

CMP3841 Wireless and Mobile Computing Semester Project

Group 3

Book Library RESTful API with Node.js and Express Documentation

Project Members:

221196994 Nuunyango F.

221141200 Linus L. J.

220070113 Ekandjo A.

221128530 Kalihonda M.

Overview

The Book Library API is a RESTful service built with Node.js and Express.js that allows users to manage a collection of books. It supports full CRUD operations and filtering.

- Base URL: *http://localhost:3000*
- Data format: JSON
- Storage: In-memory (non-persistent)

Features

- Create, read, update, and delete books
- Filter books by author, genre, or year
- Input validation middleware
- Centralized error handling
- UUID-based book identification

Book Object Structure

```
{  
  "id": "uuid",  
  "title": "string",  
  "author": "string",  
  "year": 2020,  
  "genre": "string",  
  "createdAt": "ISO date",  
  "updatedAt": "ISO date"  
}
```

Endpoints

1. Health Check

GET /:

- Returns API status and available endpoints.

Response

```
{
  "message": "Book Library API is running!",
  "version": "1.0.0"
}
```

2. Get All Books

GET /books:

- Retrieve all books with optional filters.

Query Parameters

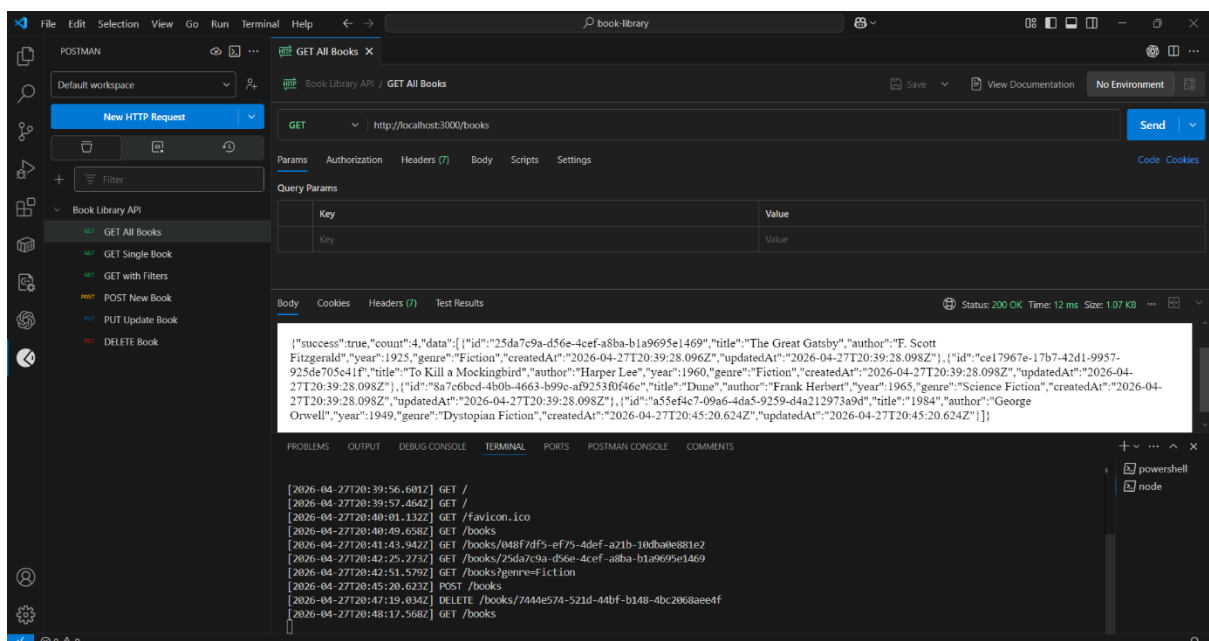
- author (string)
- genre (string)
- year (number)

Example

GET /books?author=orwell&genre=dystopian

Response

```
{
  "success": true,
  "count": 1,
  "data": [ ... ]
}
```



3. Get Book by ID

GET /books/:id

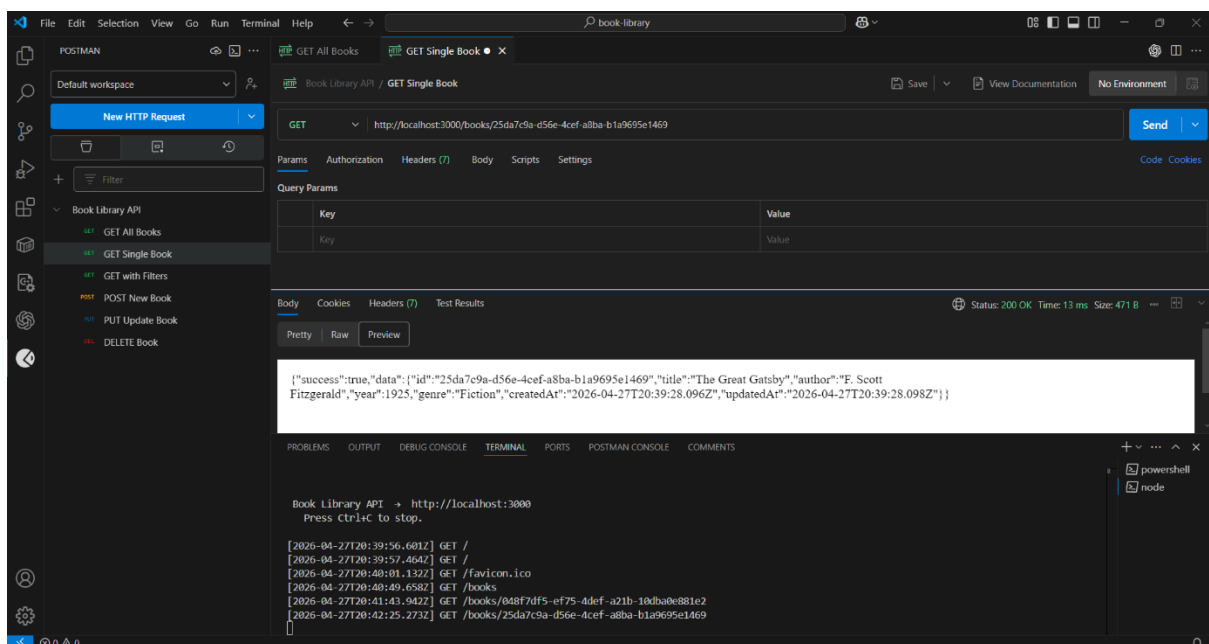
- Retrieve a single book by its UUID.

Response (Success)

```
{
  "success": true,
  "data": { ... }
}
```

Response (Not Found)

```
{
  "success": false,
  "message": "Book 'id' not found."
}
```



4. Create a New Book

POST /books

- Creates a new book.

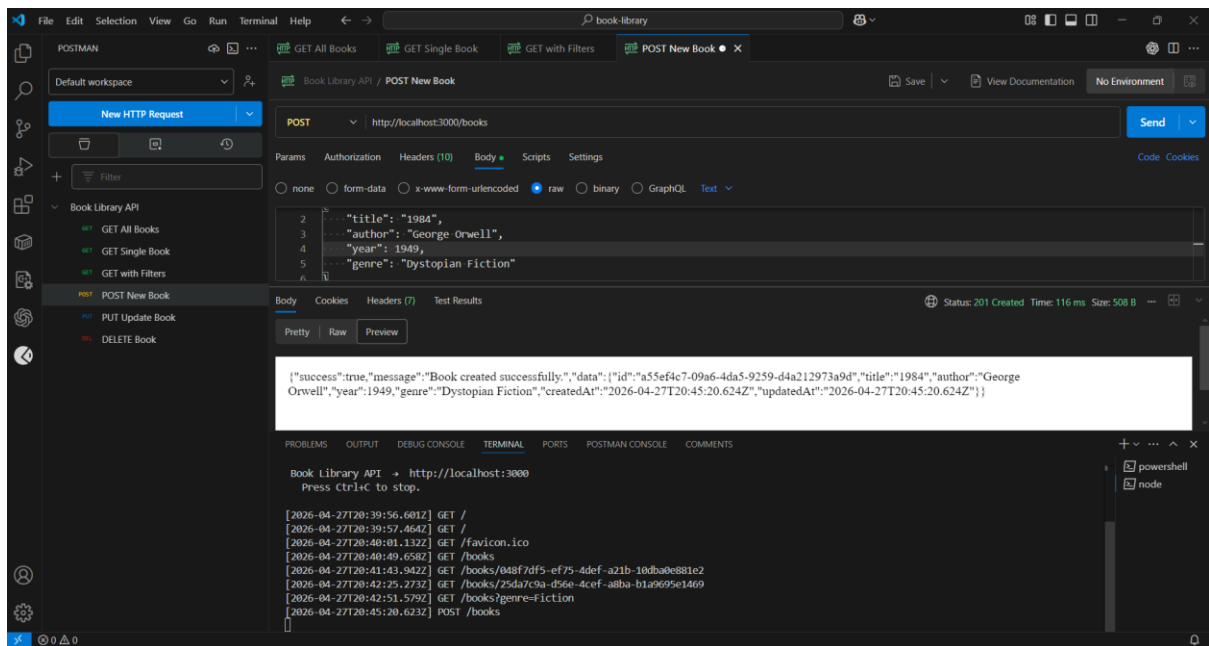
Request Body

```
{
  "title": "1984",
  "author": "George Orwell",
  "year": 1949,
}
```

```
"genre": "Dystopian Fiction"
}
```

Response

```
{
  "success": true,
  "message": "Book created successfully.",
  "data": { ... }
}
```



5. Update a Book

PUT /books/:id

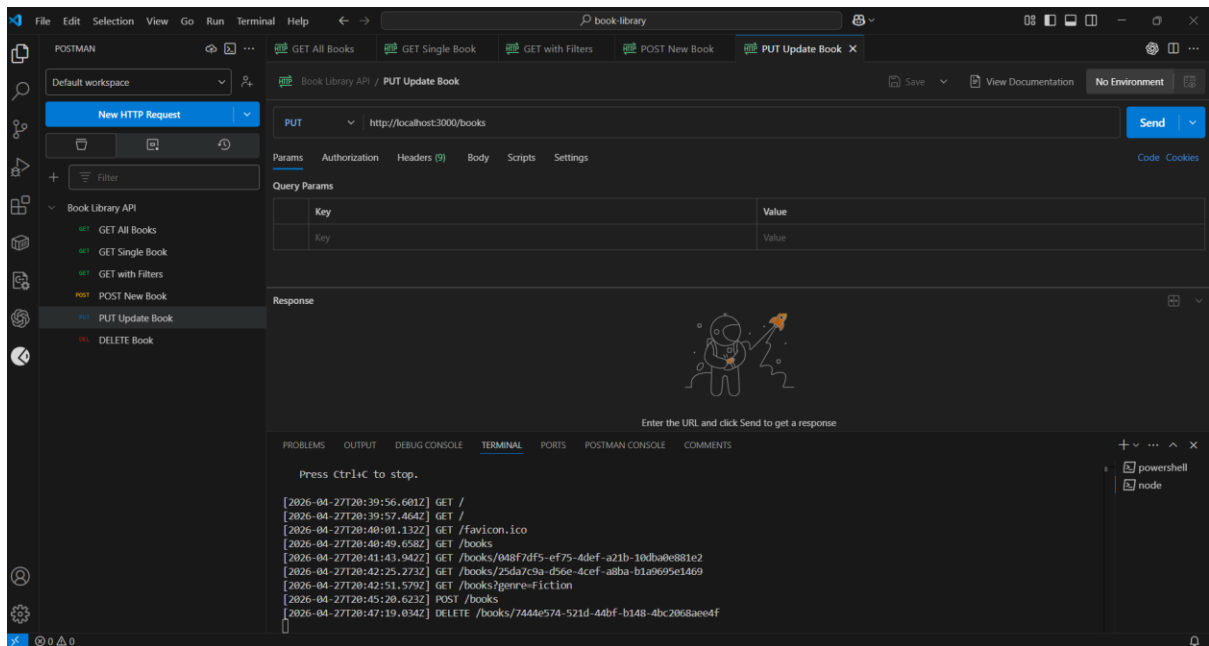
- Updates an existing book (partial updates allowed).

Request Body Example

```
{
  "genre": "Non-Fiction"
}
```

Response

```
{
  "success": true,
  "message": "Book updated successfully.",
  "data": { ... }
}
```



6. Delete a Book

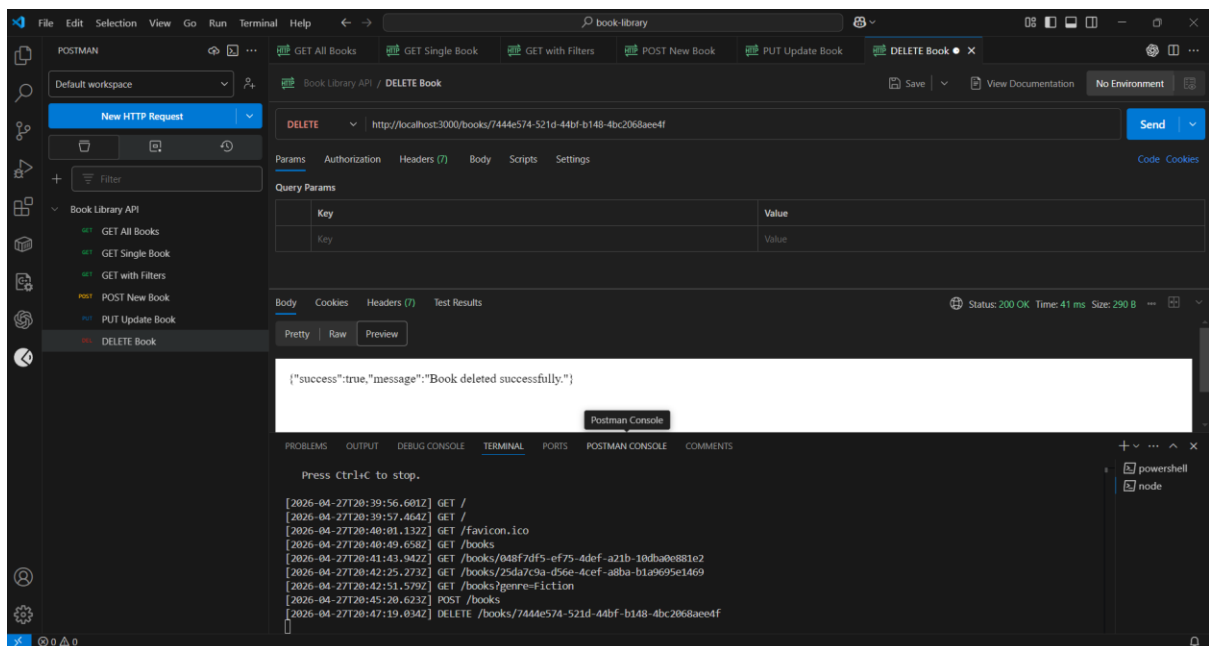
DELETE /books/:id

- Deletes a book by ID.

Response

```

{
  "success": true,
  "message": "Book deleted successfully."
}
  
```



Middleware

Validation Middleware

- validateBook: Ensures required fields are present for creation
- validatePartialBook: Allows partial updates for PUT requests

Error Handler

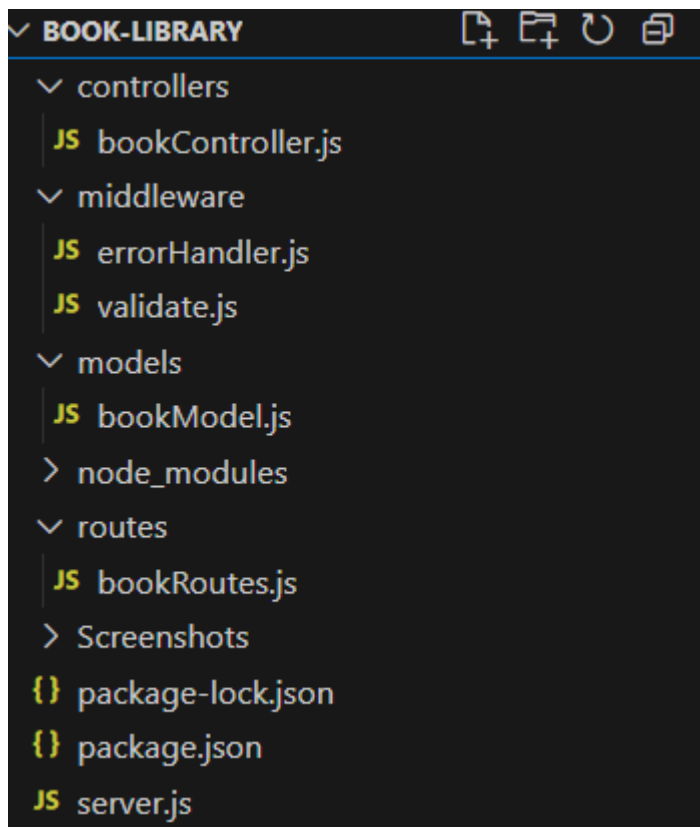
Centralized error handler returns structured error responses:

```
{  
  "success": false,  
  "message": "Error description"  
}
```

Status Codes

Code	Meaning
200	OK
201	Created
400	Bad Request
404	Not Found
500	Server Error

Project Structure



How to Run

npm install

npm start

Server runs at: *http://localhost:3000*